

Desarrollador Backend Java Springboot

Prueba Técnica

1. Descripción del Escenario

Duración estimada: máximo 4 días calendario

Modalidad: entrega individual, en repositorio Git (GitHub/GitLab, público o con acceso al evaluador)

1. Contexto

Se debe construir un microservicio que exponga una **API REST para un sistema de gestión de reservas de espacios de coworking** (salas de reuniones, puestos de trabajo, etc.).

Se requiere una base de código lista para producción, no un prototipo: se evaluará el dominio técnico de Spring Boot, no solo que "funcione".

2. Requisitos funcionales

1. **Gestión de espacios:** CRUD de espacios (nombre, tipo, capacidad, ubicación, tarifa/hora).
2. **Gestión de usuarios:** registro y autenticación de usuarios con roles ADMIN y USER.
3. **Reservas:**
 - Un usuario USER puede crear, consultar y cancelar sus propias reservas.
 - No se permiten reservas solapadas para el mismo espacio.
 - Un ADMIN puede ver y gestionar todas las reservas.
4. **Notificación asíncrona:** al confirmar una reserva, se debe simular el envío de un correo (puede ser un log o un mock) **sin bloquear** la respuesta HTTP.
5. **Reporte:** endpoint que devuelva la ocupación (%) de cada espacio en un rango de fechas, con resultados cacheados.
6. **Validación de pago externa:** al confirmar una reserva, se debe invocar un servicio externo simulado (mock endpoint o WireMock) que valide el método de pago. Este servicio debe tratarse como potencialmente inestable/lento.

3. Requisitos técnicos obligatorios (esto es lo que se evalúa)

El candidato debe **justificar en el README** por qué usó cada uno de estos elementos, no solo incluirlos:

- **Spring Boot 3.x + Java 17+**
- **Spring Data JPA** con relaciones bien modeladas y uso de @Query/Specifications donde aplique (evitar N+1).
- **Spring Security** con JWT (autenticación y autorización basada en roles).
- **Validación** con jakarta.validation (Bean Validation) y manejo centralizado de errores con @ControllerAdvice.
- **Spring Boot Actuator** expuesto con al menos health, info y métricas básicas.
- **Configuración por perfiles** (application-dev.yml, application-prod.yml) usando @ConfigurationProperties (no @Value disperso).
- **Caché** con @Cacheable/@CacheEvict para el endpoint de reportes.
- **Procesamiento asíncrono** con @Async (o eventos de dominio con ApplicationEventPublisher) para la notificación.
- **Manejo transaccional** correcto (@Transactional) en la lógica de reservas, garantizando consistencia ante solapamientos.
- **Documentación OpenAPI/Swagger** autogenerada.
- **Testing:** unitarios (Mockito) + al menos 2 pruebas de integración (@SpringBootTest con Testcontainers o base en memoria).
- **Dockerfile** funcional y, opcionalmente, docker-compose.yml con la base de datos.
- **Circuit Breaker** con **Resilience4j** (spring-cloud-starter-circuitbreaker-resilience4j) sobre la llamada al servicio externo de validación de pago. Debe incluir:
 - Configuración de umbrales (tasa de fallos, tiempo en estado abierto, etc.) vía application.yml.
 - Un método fallback que defina un comportamiento razonable cuando el circuito esté abierto (ej. dejar la reserva en estado PENDING_PAYMENT en lugar de fallar la petición).
 - Exposición del estado del circuito a través de Actuator (/actuator/health, /actuator/circuitbreakers).
- **Patrón de diseño de comportamiento (GoF):** aplicar al menos uno de forma explícita y justificada en el dominio, por ejemplo:
 - **Strategy** para el cálculo de tarifas (distintas políticas de precio según tipo de espacio, horario o duración).
 - **State** para el ciclo de vida de una reserva (PENDING, CONFIRMED, CANCELLED, COMPLETED) y sus transiciones válidas.

- **Observer** (puede apoyarse en ApplicationEventPublisher/@EventListener) para reaccionar a eventos de dominio (ej. reserva confirmada → notificar, actualizar reporte, etc.).
- **Chain of Responsibility** para la cadena de validaciones antes de confirmar una reserva. El candidato debe explicar en el README qué patrón usó, por qué, y qué problema concreto resuelve frente a una solución con if/else o switch.

4. Requisitos no funcionales / arquitectura

- Arquitectura en capas clara (controller / service / repository / dto / mapper).
- Uso de DTOs (no exponer entidades JPA directamente).
- Manejo de excepciones de negocio propias (ej. OverlappingReservationException) con códigos HTTP correctos.
- Base de datos: Postgress, incorporar instancia en docker compose.

5. Entregables

1. Repositorio Git con historial de commits legible (no un único commit "final").
2. README.md con:
 - Instrucciones de ejecución que incluya la base de datos postgres, preferible usar Docker compose. La solución debe levantar sin errores para ser calificada.
 - Decisiones de diseño y trade-offs tomados dado el límite de tiempo.
 - Qué se dejó fuera de alcance y qué se haría con más tiempo.
3. Colección Postman o archivo .http con ejemplos de uso.

7. Criterios de evaluación

** Siempre y cuando la solución levante correctamente.

Criterio	Peso
Uso idiomático y justificado de características de Spring Boot	25%
Calidad del modelado de dominio y manejo de concurrencia/transacciones	15%
Resiliencia (Circuit Breaker bien configurado y con fallback coherente)	10%
Aplicación correcta y justificada del patrón de diseño de comportamiento	15%

Criterio	Peso
Cobertura y calidad de tests	10%
Seguridad (auth/autorización correctamente implementada)	15%
Claridad de código, arquitectura y documentación	5%
Dockerización / facilidad de ejecución	5%

7. Restricciones

- **Tiempo máximo: 4 días.** Se espera que el candidato priorice; es preferible un alcance reducido pero sólido a un alcance completo pero frágil.
- No se requiere frontend.
- Se permite usar librerías adicionales siempre que se justifiquen (no reemplazar el propósito de la prueba, ej. no usar un generador de CRUD completo).
- La solución deberá ser presentada en una entrevista técnica tal como se entregó en el repositorio.

*** Buena Suerte ***